

Demeter

API Sandbox for Chrome

Technical User Manual

Product	Demeter API Sandbox (Chrome extension, Manifest V3)
Version	0.2 (modular architecture)
Platform	Google Chrome 114 or later, with the side panel feature
Audience	Frontend developers and QA testers; later chapters cover maintainers

Contents

1 Introduction

2 Installation

3 Quick Start

4 The Side Panel

5 Rules Reference

6 URL Matching

7 Presets

8 Logs

9 Common Workflows

10 Architecture

11 Development

12 Known Limitations

13 Troubleshooting

Appendix A Storage Keys and Internal Commands

1 Introduction

Demeter is a Chrome extension that lets you mock API responses directly in the browser. You define **rules** in a side panel. When a page makes a matching *fetch* or *XMLHttpRequest* call, Demeter answers it with a response you control: a custom status code, headers, body, and an artificial delay.

Everything runs locally in your browser tab. There is no proxy to configure and no backend change to make, and the real server never sees a mocked request. Mocks affect only you; other users of a live site are not impacted.

1.1 Typical uses

Scenario	Example
Build the UI before the API exists	Return a fake <code>/api/products</code> payload while the backend is unfinished
Test loading states	Delay a response by 3000 ms to check spinners and skeletons
Test error handling	Force a 500 or 503 and confirm the app shows a sensible message
Test bad data	Return malformed or empty JSON and confirm the app does not crash
Reproduce a bug	Capture a real request in Logs, convert it to a mock, then edit the body

1.2 How it works

1. You create a rule in the side panel: which address to catch and what answer to give.
2. Demeter watches the requests the page makes.
3. When a request matches a rule, Demeter waits for the delay (if any) and returns your response.
4. If nothing matches, the request goes to the real network unchanged.
5. Every request appears in the Logs tab, marked as mocked or real.

2 Installation

Demeter is loaded as an unpacked extension from the **dist/** folder. This folder, not the project root, contains the `manifest.json` that Chrome needs.

2.1 Requirements

- Google Chrome 114 or later (the build targets chrome114)
- Node.js and npm, to build the extension from source

2.2 Build

```
npm install
npm run build      # bundles src/ into dist/
# or: npm run dev  # rebuild on file changes
```

2.3 Load into Chrome

1. Open `chrome://extensions`.
2. Turn on **Developer mode** (top right).
3. Click **Load unpacked** and select the **dist** folder.
4. Click the Demeter toolbar icon (or find it under the puzzle-piece menu) to open the side panel.

Tip. If Chrome reports "Manifest file is missing or unreadable", you selected the wrong folder. Select **dist**.

2.4 Updating after code changes

Run `npm run build`, then click the reload icon on the Demeter card in `chrome://extensions`. Reload any tabs you want to test, because tabs opened before the reload keep running the old scripts.

3 Quick Start

This walkthrough mocks a slow, failing endpoint in under a minute.

1. Open the side panel and click **+ New Rule**.
2. Set **URL pattern** to the endpoint you want to mock, for example `*/api/users*`.
3. Choose a **Status** (for example 500) and/or a **Delay** (for example 2000).
4. Edit the **Response body** if needed, then click **Save**.
5. Reload the page and trigger the request.
6. Open the **Logs** tab. Your request should appear with a mocked indicator.

Important. Rules apply to every site and tab. Use specific URL patterns so you do not mock requests you did not intend to.

4 The Side Panel

The side panel is the whole user interface. Its areas are:

Area	Purpose
Header toggle	Master on/off switch for all mocking. When off, every request goes to the real network and no rules are deleted.
Preset bar	Switch, save, or delete presets (see Chapter 7).
Mocks tab	Create, edit, duplicate, delete, and toggle rules.
Logs tab	Live request list, per-rule filter, Convert to Mock, Clear Logs.
Toasts	Brief messages at the top that confirm saves and deletes, and report storage errors.

4.1 Rule cards

Each rule appears as a card. The summary row shows the rule's **name**, its method, and an enable switch. Hover over the name to see the URL pattern. Click the card to expand the editor. The enable switch saves instantly; you do not need to open the card or click Save.

The editor buttons are **View Logs**, **Duplicate**, **Delete**, and **Save**.

4.2 Validation

Invalid JSON in Headers or Response body, a bad Delay value, or an empty URL pattern shows an inline error under that field, and your typed text is kept. If two fields are wrong, both errors show.

5 Rules Reference

A rule says: *when a request matches this URL and method, answer with this response instead.*

Field	Type	Default	Description
Name	text	New rule	Label shown on the rule card.
URL pattern	glob	(empty)	What to match (Chapter 6). An empty pattern is rejected on save and never matches.
Method	select	*	* (any), GET, POST, PUT, PATCH, or DELETE.
Status	select	200	One of 200, 201, 400, 401, 403, 404, 500, 503.
Delay (ms)	number	0	Wait before responding. Must be a number, 0 or greater.
Headers	JSON object	{"Content-Type": "application/json"}	Sent with the mocked response. Must be a plain object, not an array or string.
Response body	text	{}	Returned as-is.
Enabled	toggle	on	Per-rule switch, saved instantly.

5.1 Example rule

Return a 503 with a JSON error after a 2-second wait for any request to the users API:

```
Name:      Users API down
URL pattern: */api/users*
Method:    GET
Status:    503
Delay (ms): 2000
Headers:    {"Content-Type": "application/json"}
Body:      {"error": "Service unavailable"}
```

6 URL Matching

Patterns are **globs**, not regular expressions. * matches any run of characters. All other special characters are treated literally.

Three rules govern matching:

- **Whole URL.** The pattern must match the entire URL, including the query string. A pattern that ends without * will miss URLs with extra query parameters.
- **First match wins.** Rules are checked from the top of the list down. New rules are added at the top.
- **Method** * matches every HTTP method.

Pattern	Matches	Does not match
*	every request	none
/api/users	/api/users, /api/users/42?x=1	/api/orders
https://localhost:3000/api/v1/products/*	any product path on that host	other hosts
/rest.php/v1/search/title?q=cat	queries starting with cat	?q=c, ?q=ca

Common trap. Search boxes send a request per keystroke (q=c, q=ca, q=cat). A pattern pinned to one query only catches the last. Use `*/rest.php/v1/search/title*` to catch them all.

7 Presets

A preset is a named snapshot of your rule set, such as "Happy Path", "Slow 3G", or "Error States". Presets let you switch between test scenarios in one click.

Action	What happens
Save as Preset	Stores the current rules under a name and makes it the active preset.
Select a preset	Replaces your working rules with that snapshot and clears the logs.
Edit while active	Every rule edit also updates the active preset's snapshot, so switching away and back never loses changes.
Select Custom (unsaved)	Detaches the preset label but keeps your current rules. Also clears the logs.
Delete	Removes the active preset. Your loaded rules stay in place.

8 Logs

The Logs tab shows every fetch and XHR request the page makes, mocked or real. Each entry shows the method, status, URL, and whether it was mocked or real.

- Only the **latest 40** entries are kept, shared across all tabs. Older ones drop off.
- Response bodies are truncated to 2,000 characters in the log.
- **View Logs** on a rule card filters the list to that rule's mocked traffic. A chip at the top shows the active filter; click **Show all rules** to clear it. Real requests belong to no rule, so they appear only in the unfiltered view.
- **Convert to Mock** turns a real request into a new rule, pre-filled with its URL, method, status, and captured body.
- **Clear Logs** empties the list. Switching presets also clears it.

Tip. Convert to Mock copies the URL exactly, with no wildcard. Replace dynamic parts such as IDs with * before relying on the rule.

8.1 Master toggle

The switch in the header disables all interception without deleting any rules. Use it to compare behavior with and without Demeter.

9 Common Workflows

9.1 Test a loading state

1. Create a rule for the endpoint with Status 200 and Delay 3000.
2. Reload the page and trigger the request.
3. Confirm the spinner or skeleton appears and then resolves.

9.2 Test an error state

1. Create a rule with Status 500 (or 503) and a JSON error body.
2. Trigger the request.
3. Confirm the app shows a user-friendly message rather than a blank screen.

9.3 Capture a real response and edit it

1. Browse the page normally so the request appears in Logs.
2. Click **Convert to Mock** on that entry.
3. Replace dynamic parts of the URL with *.
4. Edit the body (for example, make a list empty, or remove a field) and click Save.
5. Reload and observe how the app handles the altered data.

9.4 Build a set of scenarios

1. Create rules for a "Happy Path" and click **Save as Preset**.
2. Edit rules to simulate failures and save them as a second preset, such as "Error States".
3. Switch between presets from the dropdown during testing.

10 Architecture

This chapter is for maintainers. Chrome extensions run code in separate, isolated contexts. Demeter uses four:

Context	File (in dist/)	Role
Background service worker	background.js	The only owner of state. Reads and writes chrome.storage.
Side panel	sidepanel.js	The UI. Sends commands to the background; never writes storage directly.
Content bridge (isolated world)	content-bridge.js	Relays rules from storage to the page and forwards page logs to the background.
Content inject (MAIN world)	content-inject.js	Patches fetch and XMLHttpRequest inside the page. Has no chrome.* access.

Both content scripts run at *document_start* in all frames, so they patch the network APIs before the page's own code runs, including inside iframes.

10.1 Data flow

```
Side panel --command--> Background --writes--> chrome.storage.local
                                     |
                                     storage.onChangeed
                                     v
Page (MAIN world) <--postMessage-- Content bridge (isolated world)
  fetch / XHR patch                |
  | matched? -> mocked response    |
  +-- log entry --postMessage--> bridge --> Background --> storage (logs)
```

1. You save a rule; the side panel sends a command to the background.
2. The background writes to `chrome.storage.local`.
3. The bridge sees the storage change and posts the new rules to the page.
4. The patched fetch/XHR checks each outgoing request against the rules.
5. On a match it waits for the delay, then returns the mocked response. Otherwise the real request proceeds.
6. Either way, a log entry is posted back through the bridge to the background.

10.2 Design rules

1. Only **background/** writes to `chrome.storage`. Everyone else sends a command defined in `shared/messages.js`.
2. UI views never import each other. `sidepanel/main.js` wires them together through hooks.
3. The service worker is stateless (MV3 may stop it when idle), so all state lives in storage.
4. Page-originated data is untrusted. `logs.service` whitelists and clamps every log entry.
5. Writes are serialized through a mutex so simultaneous messages from many tabs cannot overwrite each other.

10.3 Project layout

```

demeter/
|-- src/
|   |-- manifest.json
|   |-- shared/      schema.js, messages.js, page-protocol.js, matcher.js
|   |-- background/  index.js, router.js, storage.js, services/
|   |                  (rules, presets, logs, settings, state)
|   |-- content/     bridge.js, inject/ (state, fetch-patch, xhr-patch,
|   |                  logger, utils)
|   |-- sidepanel/    sidepanel.html/css, main.js, api.js, state.js, ui/
|-- dist/             build output - load this folder in Chrome
|-- tests/            services.test.mjs
|-- docs/             documentation and historical notes
|-- build.mjs         esbuild bundler
`-- package.json

```

11 Development

Command	What it does
<code>npm run build</code>	Bundles each context into <code>dist/</code> as one self-contained script per entry (MV3 content scripts cannot use ES imports).
<code>npm run dev</code>	Rebuilds on file changes. You still need to reload the extension in Chrome.
<code>npm test</code>	Runs the unit tests with Node's built-in test runner.

The tests cover concurrent log writes, log capping and sanitizing, preset snapshot syncing, Custom/preset switching, dangling preset IDs, and the URL matcher.

12 Known Limitations

- **Only fetch and XMLHttpRequest are intercepted.** WebSockets, Server-Sent Events, and requests made by service workers are not. A PWA that routes API calls through a service worker will not be mocked.
- **Mocked responses only.** Passthrough requests are never modified: no header changes and no URL redirects.
- **Fixed response codes.** The Status dropdown offers a fixed list. There is no random or percentage-based failure.
- **XHR mocking is partial.** Status, ready state, response, and responseText are set. Other properties such as getAllResponseHeaders are not emulated.
- **Rules are global.** They apply to every site and tab, not per site.
- **Logs are shared and short.** The latest 40 entries across all tabs, not per tab.
- **Glob matching only.** Regular expressions are not supported, by design.
- **No import/export** of rules yet.
- **No custom icon.** Chrome shows its default placeholder.

13 Troubleshooting

Symptom	Likely cause and fix
"Manifest file is missing or unreadable"	You loaded the project root. Load dist/ instead.
Mock is not applied	Check the master toggle and the rule toggle, confirm the method matches, and reload the tab (old tabs keep old scripts).
Rule matches some requests but not others	Pattern too narrow. Add a trailing * and check the query string (Chapter 6).
Real data still appears	The request may not use fetch/XHR (service worker, WebSocket), or the UI may be showing browser autofill or the site's own saved history.
Rules gone after installing Demeter	A new unpacked extension gets fresh storage, so rules from a previous extension do not carry over. Recreate them.
Two similar extensions interfering	Remove the old one in chrome://extensions.
Mocking stopped in an open tab after reloading the extension	Expected. Refresh the tab.
Nothing in Logs	Reload the page and check that the panel is showing the right context (Custom vs a preset).

13.1 Finding errors

Open chrome://extensions and click **Errors** on the Demeter card, or inspect the side panel with DevTools. Warnings from the page-side scripts are prefixed [Demeter] in the page console.

Appendix A Storage Keys and Internal Commands

Storage keys (chrome.storage.local)

Key	Contents
demeterRules	The active working rule set
demeterEnabled	Master toggle state
demeterPresets	Saved preset snapshots
demeterActivePresetId	Currently selected preset (empty = Custom)
demeterLogs	Captured requests (max 40)

Internal commands

The side panel and bridge talk to the background only through these commands. Each returns { ok: true, data } or { ok: false, error }.

Area	Commands
State / settings	state/get, settings/setEnabled
Rules	rules/add, rules/update, rules/remove, rules/duplicate
Presets	presets/save, presets/activate, presets/delete
Logs	logs/append, logs/clear